TEI Publisher 10: Tips and tricks

e-editiones community meetup April 7th, 2026

App Customization

Before adding code, ask:

> *Can this be done in* `config.json`*?*

- All profiles come with configuration settings
- JSON Schema (`jinks.json`) is the ultimate source of truth
- Check **Merged Configuration** dropdown in Jinks for all config options

Report Back

If there's no configuration setting yet, ask:

> **Should there be one?**

- If yes, open an issue / PR or start a discussion on slack

Don’t Fight the Generator

Avoid changing files generated by Jinks

Instead: add your own CSS, scripts, and API pieces as **separate files**, and **register** them in `config.json` or in **HTML template frontmatter**.

Respect the conventions

Store your additions where they belong:

- XQuery modules in `modules`
- HTML page templates and blocks in `templates` or its subdirectories
- general CSS in `resources/css`,
- ODDs and ODD-related css in `resources/odd`,
- JS in `resources/scripts`,
- images in `resources/images`

NB conventions are the same as in earlier Publisher versions

CSS

Use `styles` array to add custom CSS files

```
{
    "styles" : [ "resources/css/shakespeare-theme.css" ]
}
```

Best practice: prefix CSS filename with project abbrev and save in `resources/css`.

Javascript

Use `script.custom`:

```
{
    "script": {
        "custom": [
            "resources/scripts/shakespeare-facets.js"
        ]
    }
}
```

Open API Endpoints (XQuery)

Add `.json` and corresponding `.xql` implementation to `modules` and register in `config.json`:

```
{
    "api" : [
        {
            "spec" : "shakespeare-api.json",
            "prefix" : "shakes",
            "id" : "https://e-editiones.org/api/shakespeare",
            "path" : "shakes-api.xql"
        }
    ]
}
```

HTML Templates

- Default template defined in `defaults.template`
- Start by copying `templates/pages/basic.html` for document views
- Respect the template inheritance hierarchy
- Avoid directly changing `content.html` or `base.html`
- Stick to the default page layout areas for arranging content on the page

Using Frontmatter

Many settings can be controlled at page level, in HTML frontmatter:

```
<template>
    ---json
    {
        "templating": {
            "extends": "templates/layouts/base.html"
        },
        "styles": [ "resources/css/landing-page.css" ]
    }
    ---
    <p>Page content</p>
</template>
```

Custom HTML Blocks

Inject custom HTML blocks into pages with `templating.use`:

```
"templating" : {
    "use" : [ "templates/iiif-blocks.html" ]
}
```

Managing conflicts

- Sometimes you need to overwrite a file, e.g. add or remove landing page sections in `templates/index.html`
- If you don't want to be reminded about this *expected* conflict, add it to the `ignore` list in `config.json`:

```
{
    "ignore" : [ "templates/index.html" ],
}
```

Development Workflow

1. create base application via Jinks
2. synchronize to local directory
3. commit to version control

When to Regenerate?

- `config.json` changed
- CSS related to webcomponents changed
- color palette changed

When to Sync?

- in particular:
  - after editing the ODD with the ODD editor
  - after regenerating the app with Jinks
- in general:
  - after each change to the database you don't have locally

Sync Options: eXist-db -> local directory

- `sync` action in Jinks
  - writes to local directory on server machine
  - *docker*: inside container!
- `Download` .xar and unpack
- `Files` tab in Jinks (right click on file to download)

Sync Options: local directory -> eXist-db

- eXist-db plugin for Visual Studio Code and similar editors
- always **rebuild** `.xar` with `ant` and redeploy
- use xst command line tool

Golden Rules

- always commit `.jinks.json` and `context.json`
- never change `context.json` directly (it's autogenerated every time)

Caveat: synchronizing binary files

Binary files can be unnecessarily changed when synchronizing between the database and the local directory.

FONTS can be entirely broken in the synchronization, only change them through the full `xar` rebuild and deploy

Upgrading an old app (before v10)

There's no direct update path – app needs to be recreated.

1. **Create** new app in Jinks
   - **abbreviation** and **Unique Identifier** must be different
   - **add profiles** as needed, but start with minimum set first
2. **Download** the `.xar` package and unpack into new folder

Copy data

1. copy any data from the source app's `data` subdirectory into the target app's `data`
2. compare `collection.xconf` files and check if changes are necessary

Copy ODDs

1. copy any custom ODD, excluding `teipublisher.odd`, `tei_simplePrint.odd`, `docx.odd`, `teipublisher_odds.odd`
2. make sure that all custom ODDs are registered in `config.json`

```
{
    "odds" : [ "shakespeare.odd" ],
    "defaults" : {
        "odd" : "shakespeare.odd"
    }
}
```

3. Run **Fix ODDs** action to regenerate ODD configuration

Repair ODDs

1. check ODDs extending `teipublisher.odd` (`source` attribute on `<schemaSpec>`):
   - for each `<elementSpec>` overwriting one in `teipublisher.odd`: make sure that the `mode` attribute on the `<elementSpec>` is set to `"change"`, not `"add"`
2. check where `$parameters?header='short'` is used in a `predicate` attribute on a `model`. Replace with `$parameters?display='browse'`

Copy and adjust HTML templates

- Make sure `defaults.template` in `config.json` references correct template
- HTML templates need to be adjusted to new templating system
- Start from `templates/pages/basic.html` for document views
- Observe template inheritance hierarchy!